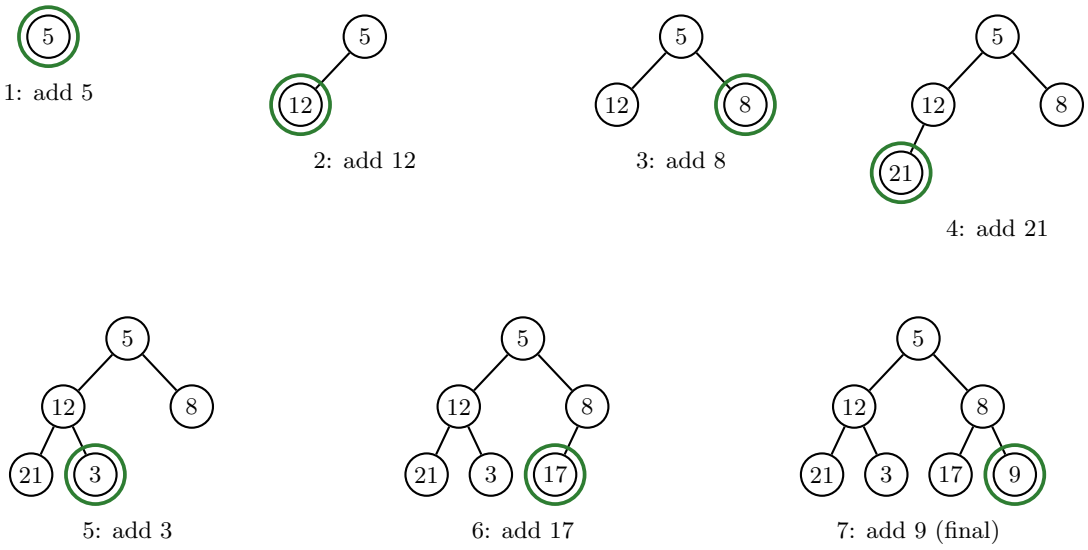
Problem 1: Creating the Tree

$arr = \{5, 12, 8, 21, 3, 17, 9\}$, $n = 7$. ○ node added / value moved ○ deletion target

(a)

Step	Element	Index i	Parent index $(i-1)/2$	Parent value	Left / Right	Rule(s)
1	5	0	—	—	root	C1
2	12	1	$(1-1)/2 = 0$	5	Left (i odd)	C2, C3
3	8	2	$(2-1)/2 = 0$	5	Right (i even)	C2, C4
4	21	3	$(3-1)/2 = 1$	12	Left (i odd)	C2, C3
5	3	4	$(4-1)/2 = 1$	12	Right (i even)	C2, C4
6	17	5	$(5-1)/2 = 2$	8	Left (i odd)	C2, C3
7	9	6	$(6-1)/2 = 2$	8	Right (i even)	C2, C4



Final tree (level order): 5 12 8 21 3 17 9

(b) 21, 3, 17, 9 (indices 3–6) have NULL children. Rule **C5**: their child indices $2i+1$, $2i+2$ are 7–14, all $\geq n = 7$, so those children do not exist.

(c)

Root	5
Leaves	21, 3, 17, 9
Internal nodes	5, 12, 8
Edges	6 $(n-1)$
Height	2 $(5 \to 12 \to 21)$
Depth of 17	2
Ancestors of 3	12, 5
Descendants of 12	21, 3
Subtree at 8	$\{8, 17, 9\}$
Path 5 to 9	$5 \to 8 \to 9$, length 2

(d)

Max nodes on level 2	$2^2 = 4$; level 2 has 4 ✓
Max nodes for $H = 2$	$2^3 - 1 = 7 = n$ ✓
Leaves = two-child + 1	5, 12, 8 $\Rightarrow 3 + 1 = 4$ ✓
Min height, $N = 7$	$\lfloor \log_2 7 \rfloor = 2 =$ height ✓

Both full and complete. C6: creation fills level by level, left to right, so the tree is always complete. Since $n = 7 = 2^{H+1} - 1$, every level is filled, so it is also full.

Problem 2: Traversing the Tree

(a–c) Each cell gives the node visited and why it is visited at that moment.

#	(a) Preorder (N L R)	(b) Inorder (L N R)	(c) Postorder (L R N)
1	**5**: root; N, then L (12), R (8)	**21**: L is NULL, so N	**21**: L, R NULL, so N
2	**12**: N, then L (21), R (3)	**12**: L (21) done, so N	**3**: L, R NULL, so N
3	**21**: N; L, R NULL	**3**: L is NULL, so N	**12**: L (21), R (3) done, so N
4	**3**: N; L, R NULL	**5**: L (12) done, so N	**17**: L, R NULL, so N
5	**8**: N, then L (17), R (9)	**17**: L is NULL, so N	**9**: L, R NULL, so N
6	**17**: N; L, R NULL	**8**: L (17) done, so N	**8**: L (17), R (9) done, so N
7	**9**: N; L, R NULL	**9**: L is NULL, so N	**5**: L (12), R (8) done, so N
Rules	T2; T1, T6 at leaves	T3; T1, T6 at leaves	T4; T1, T6 at leaves
Result	**5 12 21 3 8 17 9**	**21 12 3 5 17 8 9**	**21 3 12 17 9 8 5**

(d) Level order (T5 at every row).

#	Queue before	Remove & visit	Add children (left, right)	Queue after	Output so far
1	5	5	12, 8	12, 8	5
2	12, 8	12	21, 3	8, 21, 3	5 12
3	8, 21, 3	8	17, 9	21, 3, 17, 9	5 12 8
4	21, 3, 17, 9	21	none	3, 17, 9	5 12 8 21
5	3, 17, 9	3	none	17, 9	5 12 8 21 3
6	17, 9	17	none	9	5 12 8 21 3 17
7	9	9	none	empty, stop	**5 12 8 21 3 17 9**

(e) The tree was built in level order (C6), so T5 visits the nodes in index order $0, 1, \ldots, 6$, which is *arr*. **Postorder** visits the root last: by T4 a node is visited only after both of its subtrees, and every other node is in a subtree of 5.

(f) Search 3: compares 5, 12, 8, 21, 3, so it is found after **5** comparisons. Search 4: compares all 7 nodes, so it is not found after **7** comparisons (worst case, n).

Problem 3: Adding Elements to the Tree

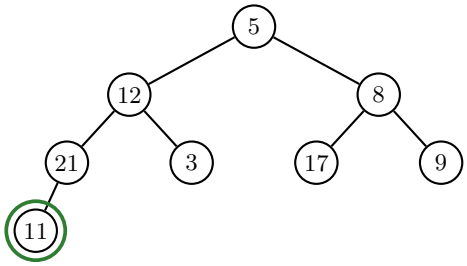
Each insertion after (a) scans in level order with a queue (I2). Children are listed as (left, right); – means NULL.

Insert	Nodes checked (left, right)	Rule per node	Parent	Side	Level order after
11	5 (12, 8), 12 (21, 3), 8 (17, 9), 21 (–, –)	I5 ×3, I3	21	Left	5 12 8 21 3 17 9 11
2	5, 12, 8 (both present), 21 (11, –)	I5 ×3, I4	21	Right	5 12 8 21 3 17 9 11 2
14	5, 12, 8, 21 (both present), 3 (–, –)	I5 ×4, I3	3	Left	5 12 8 21 3 17 9 11 2 14
6	5, 12, 8, 21 (both present), 3 (14, –)	I5 ×4, I4	3	Right	5 12 8 21 3 17 9 11 2 14 6

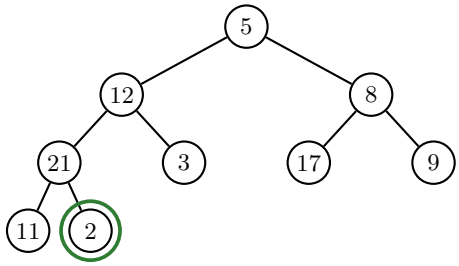
(a) The tree is empty, so 5 becomes the root (**I1**).
(b) 5, 12, 8 are full (I5); 21 has a free left position, so 11 is the **left child of 21** (I3).
(c) 21's left is now 11, so I3 fails and **I4** puts 2 on the **right of 21** (index $8 = 2(3) + 2$).
(d) Skipped by I5: **5, 12, 8, 21**. 14 is the **left child of 3** (I3).
(e) I6: index $= n = 10$; parent $(10 - 1)/2 = 4$, value 3; $10 = 2(4) + 2$, so **right of 3**. Full scan: 3 has left 14 and a free right (I4) ✓



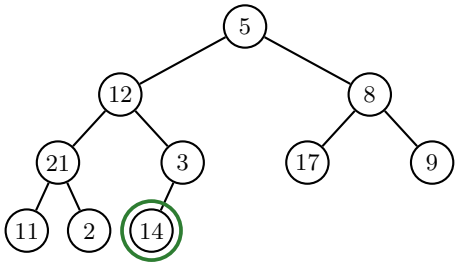
(a) insert 5



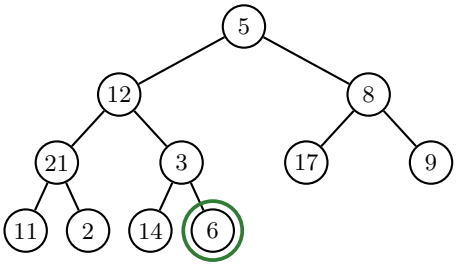
(b) insert 11



(c) insert 2



(d) insert 14



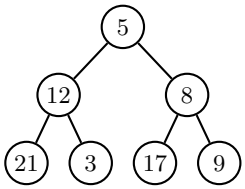
(e) insert 6

Problem 4: Deleting Elements (Every Case)

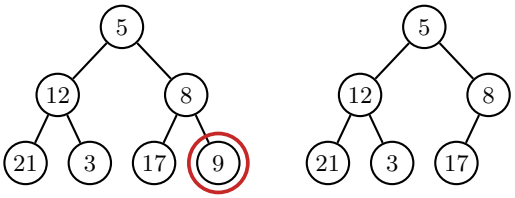
Each part starts from the original tree 5 12 8 21 3 17 9, whose deepest node is 9 (D3).

(a) Case A: delete 4. Level order checks all 7 nodes and never finds 4 (D3), so the tree is unchanged (D4).

(b) Case B: delete 9. Target = deepest (D7): nothing is copied; D6 clears 8's right link. Easiest case: no copy, no other node changes.



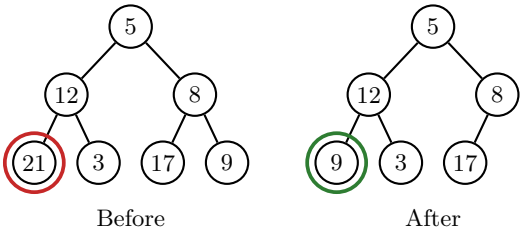
Before = after



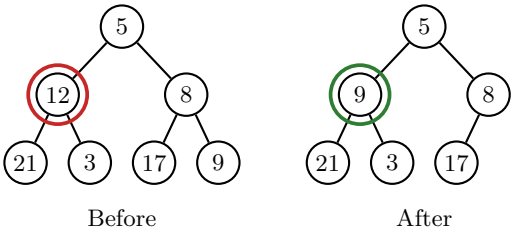
Before After

(c) Case C: delete 21. Target 21 (leaf), deepest 9. Copy 9 into 21's node (D5), remove the old 9 (D6).

(d) Case D: delete 12. Target 12 (internal), deepest 9 (D5, D6). The replacement 9 keeps 12's children: **21 (left) and 3 (right)**.

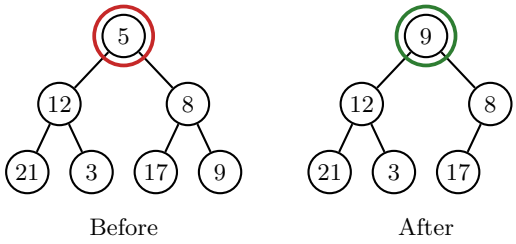


Before After



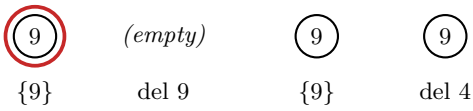
Before After

(e) Case E: delete 5. Target 5 (root), deepest 9. Copy 9 into the root (D5), remove the old 9 (D6). New root: **9**.



(f) Case F: {9}. Delete 9: the only node holds the key, so the tree becomes empty. Delete 4 on a fresh {9}: not found, so it stays {9} (D2).

(g) Case G: the tree is empty; delete 5 does nothing (D1).



(h) Removing the target directly would leave a hole: an internal target's children would lose their parent, and the array would have a gap, so the tree would no longer be complete. The deepest node is the last node in level order and always a leaf, so removing it disconnects nothing and keeps the tree complete; only one value is copied (D5) and one link cleared (D6).

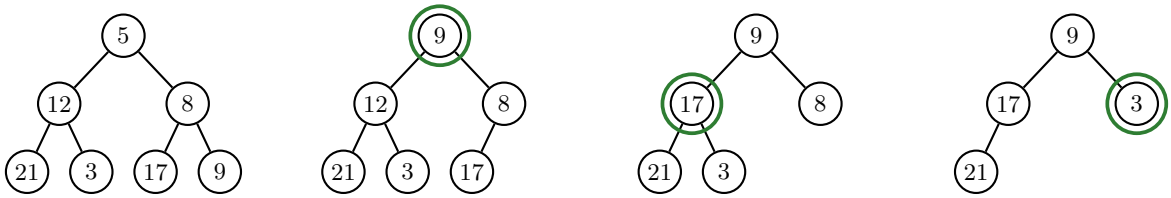
Case	Delete	Target	Deepest	Rules applied	Level order after
A	4	not found	9	D3, D4	5 12 8 21 3 17 9 (unchanged)
B	9	9	9	D3, D7	5 12 8 21 3 17
C	21	21	9	D3, D5, D6	5 12 8 9 3 17
D	12	12	9	D3, D5, D6	5 9 8 21 3 17
E	5	5	9	D3, D5, D6	9 12 8 21 3 17
F	9 / 4	9 / not found	9 / 9	D2	empty / 9
G	5	—	—	D1	empty

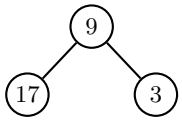
Problem 5: Deleting All Elements

(a) Delete 5, 12, 8, 21, 3, 17, 9 without resetting the tree.

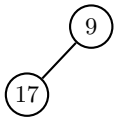
#	Delete	Tree before	Deepest	Case	Rules	Tree after
1	5	5 12 8 21 3 17 9	9	E (root)	D3, D5, D6	9 12 8 21 3 17
2	12	9 12 8 21 3 17	17	D (internal)	D3, D5, D6	9 17 8 21 3
3	8	9 17 8 21 3	3	C (leaf)	D3, D5, D6	9 17 3 21
4	21	9 17 3 21	21	B (deepest)	D3, D7	9 17 3
5	3	9 17 3	3	B (deepest)	D3, D7	9 17
6	17	9 17	17	B (deepest)	D3, D7	9
7	9	9	9	F (one node)	D2	empty

(b)





4: del 21



5: del 3



6: del 17

(empty)

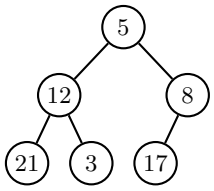
7: del 9

(c) Cases E, D, C, B (three times) and F appeared; A and G did not (every key was present and the tree was never empty at the start of a deletion). The tree stayed complete after every step, because each deletion only removes the last node in level order, so the remaining nodes fill indices 0 to $n-2$ with no gaps. Values copied: **3**.

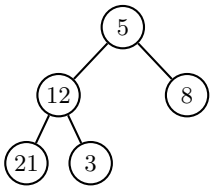
Bonus: Reverse Order

Delete 9, 17, 3, 21, 8, 12, 5 from the original tree.

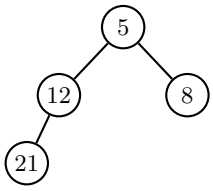
#	Delete	Tree before	Deepest	Case	Rules	Tree after
1	9	5 12 8 21 3 17 9	9	B	D3, D7	5 12 8 21 3 17
2	17	5 12 8 21 3 17	17	B	D3, D7	5 12 8 21 3
3	3	5 12 8 21 3	3	B	D3, D7	5 12 8 21
4	21	5 12 8 21	21	B	D3, D7	5 12 8
5	8	5 12 8	8	B	D3, D7	5 12
6	12	5 12	12	B	D3, D7	5
7	5	5	5	F	D2	empty



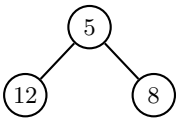
1: del 9



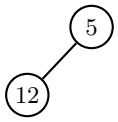
2: del 17



3: del 3



4: del 21



5: del 8



6: del 12

(empty)

7: del 5

Case B at steps 1–6 and F at step 7. This order is the simplest because reversing *arr* reverses the level order (C6), so the key deleted is always the deepest node: nothing is copied and only one leaf link is cleared (D7). Values copied (D5): **3** in the forward order, **0** in the reverse order.